CSC4202

Semester 2, 2025-2026

GROUP PROJECT PROJECT LOG

APPENDIX

Initial Project Plan (week 10, submission date: 5 June 2026)

<table>
<tr><td>Group Name</td><td>GROUP 4</td></tr>
<tr><td>Members</td><td>
<table>
<tr><td>Name</td><td>No matrix</td><td>email</td><td>Phone number</td></tr>
<tr><td>MUHAMMAD DANIAL BIN WAN NORAHSAM</td><td>222448</td><td>222448@student.upm.edu.my</td><td>013-5571850</td></tr>
<tr><td>KHAIROL RHAZMI BIN FAKHRI</td><td>223804</td><td>223804@student.upm.edu.my</td><td>0166675803</td></tr>
<tr><td>DANIEL IQBAL BIN MOHD MARJAN</td><td>223749</td><td>223749@student.upm.edu.my</td><td>0196072495</td></tr>
<tr><td>SHAH MUHAMMAD ASH-SYAKIR BIN SHAHRIL</td><td>SA00339</td><td>SA00339@student.upm.edu.my</td><td>01156821052</td></tr>
</table>
</td></tr>
<tr><td>Problem scenario description</td><td>A severe monsoon landslide on 14 November 2025 blocks the main highway (Jalan Hulu Bernam) connecting Hospital Kuala Kubu Bharu to Kampung Sungai Lui in Hulu Selangor, Selangor. A cardiac patient (Mr. Ahmad bin Ramli, 58) requires urgent ambulance evacuation. The ambulance must navigate through a fragmented network of surviving secondary logging roads, plantation tracks, and forest paths. The problem is to find the fastest route (minimum total travel time) from the hospital to the village through this surviving road network.</td></tr>
<tr><td>Why it is important</td><td>Cardiac emergencies require intervention within 90 minutes. Every minute of route delay reduces survival probability by 7–10% (Holmberg et al., 2001). An algorithmic solution guarantees the provably fastest detour route, eliminating reliance on intuition or guesswork by dispatchers under stressful, dark, and rain-affected conditions.</td></tr>
<tr><td>Problem specification</td><td>Input = Weighted undirected graph G = (V, E, w) with V = 8 nodes, E = 9 surviving road edges, each with a travel time weight in minutes. Source = Node 0 (Hospital KKB), Destination = Node 7 (Kg. Sungai Lui).
Output = Shortest path (ordered sequence of node IDs) and minimum total travel time from source to destination.</td></tr>
</table>

	Constraint = All edge weights ≥ 0 (travel times cannot be negative).
Potential solutions	1. **Sorting =** Not suitable because cannot find paths and only ranks individual items. 2. **Divide and Conquer =** Not suitable because subproblems are not independent and paths share intermediate nodes 3. **Dynamic Programming (Bellman-Ford)=** Can be choose but O(VE) is a bit unnecessarily slow for non-neg weights 4. **Greedy(naive) =** Not optimal because no global optimality guarantee on weighted graphs 5. **Graph (Dijkstra's) =** Best choice because optimal O((V + E)Log V) and also correct for all non-negative weights.
Sketch (framework, flow, interface)	**Framework =** Non-Edge Diagram **Flow =** Build graph from edge list Initialize: dist[0] =0 , dist [all other] = ∞, prev[all] =-1 Insert all nodes into min-priority queue (PQ) **MAIN LOOP: while PQ not empty** **u ← PQ.extractMin()** **if visited[u]: skip (stale entry)** **visited[u] = true** **for each (v, w) in adj[u]:** **if dist[u] + w < dist[v]:** **dist[v] = dist[u] + w** **prev[v] = u** **PQ.insert(dist[v], v)** Backtrack via prev[] from destination Output: Optimal path + Total travel time **Program interface =** Console Output

Project Proposal Refinement (week 11, submission date: 12 June 2026)

<table>
<tr><td>Group Name</td><td>GROUP 4</td></tr>
<tr><td>Members</td><td><table><tr><th>Name</th><th>Role</th></tr><tr><td>MUHAMMAD DANIAL BIN WAN NORAHSAM</td><td>Algorithm Design and Java Implementation</td></tr><tr><td>KHAIROL RHAZMI BIN MOHD FAKHRI</td><td>Documentation and Online portfolio</td></tr><tr><td>DANIEL IQBAL BIN MOHD MARJAN</td><td>Analysis , Correctness Proof and Presentation</td></tr><tr><td>SHAH MUHAMMAD ASH-SYAKIR BIN SHAHRIL</td><td>Scenario model development and documentation</td></tr></table></td></tr>
<tr><td>Problem statement</td><td>Given a weighted undirected graph G = (V, E, w) representing the post-landslide road network with non-negative weights (travel times in minutes), find the minimum-time path from Hospital Kuala Kubu Bharu (source) to Kampung Sungai Lui (destination) using Dijkstra's Algorithm.</td></tr>
<tr><td>Objectives</td><td>1. Model the post-landslide road network as a weighted graph with 8 nodes and 9 edges.
2. Implement Dijkstra's Algorithm in Java with min-priority queue for O((V+E)logV) performance.
3. Prove correctness using the Loop Invariant technique.
4. Analyse time complexity for best, average, and worst cases.
5. Demonstrate three test scenarios including a fully isolated village edge case.</td></tr>
<tr><td>Expected output</td><td>- Optimal route as a node sequence (e.g., 0→1→3→5→7)
- Minimum total travel time (55 minutes for Scenario A)
- Per-segment travel time breakdown
- Automatic re-routing if additional roads are blocked</td></tr>
<tr><td>Problem scenario description</td><td>A severe monsoon landslide on 14 November 2025 blocks the main highway (Jalan Hulu Bernam) connecting Hospital Kuala Kubu Bharu to Kampung Sungai Lui in Hulu Selangor, Selangor. A cardiac patient (Mr. Ahmad bin Ramli, 58) requires urgent ambulance evacuation. The ambulance must navigate through a fragmented network of surviving secondary logging roads, plantation tracks, and forest paths. The problem is to find the fastest route (minimum total travel time) from the hospital to the village through this surviving road network.</td></tr>
<tr><td>Why it is important</td><td>Cardiac emergencies require intervention within 90 minutes. Every minute of route delay reduces survival probability by 7–10% (Holmberg et al., 2001). An algorithmic solution guarantees the provably fastest detour route, eliminating reliance on intuition or guesswork by dispatchers under stressful, dark, and rain-affected conditions.</td></tr>
<tr><td>Problem specification</td><td>Input = Weighted undirected graph G = (V, E, w) with V = 8 nodes, E = 9 surviving road edges, each with a travel time weight in minutes. Source = Node 0 (Hospital KKB), Destination = Node 7 (Kg. Sungai Lui).
Output = Shortest path (ordered sequence of node IDs) and minimum total travel time from source to destination.</td></tr>
</table>

<table>
<tr><td></td><td>**Constraint =** All edge weights ≥ 0 (travel times cannot be negative).</td></tr>
<tr><td>Potential solutions</td><td>**1.** Sorting = Not suitable because cannot find paths and only ranks individual items.
2. Divide and Conquer = Not suitable because subproblems are not independent and paths share intermediate nodes
3. Dynamic Programming (Bellman-Ford)= Can be choose but O(VE) is a bit unnecessarily slow for non-neg weights
4. Greedy(naive) = Not optimal because no global optimality guarantee on weighted graphs
5. Graph (Dijkstra's) = Best choice because optimal O((V + E)Log V) and also correct for all non-negative weights.</td></tr>
<tr><td>Sketch (framework, flow, interface)</td><td>**Framework =** Non-Edge Diagram
Flow =
Build graph from edge list
Initialize: dist[0] =0 , dist [all other] = ∞, prev[all] =-1
Insert all nodes into min-priority queue (PQ)
MAIN LOOP: while PQ not empty
u ← PQ.extractMin()
if visited[u]: skip (stale entry)
visited[u] = true

for each (v, w) in adj[u]:
if dist[u] + w < dist[v]:
dist[v] = dist[u] + w
prev[v] = u
PQ.insert(dist[v], v)

[0: Hospital KKB] —— 8 —— [1: Pekan KKB Junction]
**(main road / **
BLOCKED) 10 18
**/ **
[2: Batang Kali] [3: Sg. Chilling]
/ \\ /
20 36 17
/ \\ |
[4: Plantation] [5: Pos Gertak] ←—(optimal)
| |
│ 10 12
| |
[6: Hulu Bernam] [7: Kg. Sungai Lui] ← DESTINATION
\\ /
—------14——------

Optimal Path (highlighted):
0 → 1 → 3 → 5 → 7` = 8 + 18 + 17 + 12 = 55 minutes

Backtrack via prev[] from destination
Output: Optimal path + Total travel time</td></tr>
</table>

	Program interface = Console Output ══════════════════════════════════ LANDSLIDE EVACUATION ROUTE OPTIMISER Source : Node 0 — Hospital Kuala Kubu Bharu Destination : Node 7 — Kampung Sungai Lui ✔ Shortest Travel Time : 55 minutes Optimal Route : 0 → 1 → 3 → 5 → 7 Segment Breakdown Node 0 → Node 1 : 8 min Hospital KKB → Pekan KKB Node 1 → Node 3 : 18 min Pekan KKB → Sg. Chilling Node 3 → Node 5 : 17 min Sg. Chilling → Pos Gertak Node 5 → Node 7 : 12 min Pos Gertak → Kg. Sungai Lui TOTAL : 55 min
Methodology	<table><tr><td>Milestone</td><td>Time</td></tr><tr><td>Scenario refinement + graph design + algorithm selection</td><td>wk10</td></tr><tr><td>Algorithm suitability review + mathematical model + pseudocode</td><td>wk11</td></tr><tr><td>Java implementation + test scenarios + debugging</td><td>wk12</td></tr><tr><td>Correctness proof + time complexity analysis + portfolio completion</td><td>wk13</td></tr><tr><td>Presentation preparation + submission + delivery</td><td>wk14</td></tr></table>

Project Progress (Week 10 – Week 14)

Milestone 1	**Scenario Development**
Date (week)	**WEEK 10 (5 June 2026)**
Description/ sketch	- Group agreed on the Landslide Medical Evacuation scenario set in Hulu Selangor. - Identified the problem as a Single-Source Shortest Path problem on a weighted graph.

	- Agreed that Dijkstra's Algorithm (Graph Paradigm) is the optimal algorithm choice. - Sketched the preliminary 8-node road network graph with estimated travel times. - Identified 9 surviving road edges and confirmed the main highway is the blocked edge. - Drafted the initial project plan and submitted on time.
Role	<table><tr><td>Member 1</td><td>Member 2</td><td>Member 3</td><td>Member 4</td></tr><tr><td>DANIAL</td><td>KHAIROL</td><td>SHAH</td><td>IQBAL</td></tr><tr><td>Scenario brainstorming and geographical setting</td><td>Initial documentation & formatting</td><td>Algorithm paradigm comparison (initial review)</td><td>Graph sketch (node & edge design)</td></tr></table>

Milestone 2	**Scenario model development and algorithm selection**
Date (Wk)	**WEEK 11 (12 June 2026)**
Description/ sketch	- Refined scenario with specific Malaysian geographical details (Hospital KKB, Jalan Hulu Bernam, Pos Gertak, Kampung Sungai Lui). - Finalized 8-node, 9-edge graph with edge weight justifications (based on distance/safe speed estimates). - Completed full algorithm suitability review (Sorting, DAC, DP, Greedy, Graph) with strengths/weaknesses table. - Derived mathematical model: objective function $\Sigma w(v_i,v_{i+1})$ minimized subject to $x_i \in \{0,1\}$ per edge. - Wrote relaxation recurrence relation: dist[v] = min(dist[v], dist[u] + w(u,v)). - Prepared pseudocode for BuildGraph, Dijkstra, ReconstructPath functions.
Role	<table><tr><td>Member 1</td><td>Member 2</td><td>Member 3</td><td>Member 4</td></tr><tr><td>DANIAL</td><td>KHAIROL</td><td>SHAH</td><td>IQBAL</td></tr><tr><td>Scenario & geography refinement</td><td>Algorithm suitability review (full table)</td><td>Mathematical model & recurrence relation</td><td>Edge weight table & graph documentation</td></tr></table>

<table>
<tr><td>Milestone 3</td><td>Code Implementation</td></tr>
<tr><td>Date (week)</td><td>WEEK 12 (19 June 2026)</td></tr>
<tr><td>Description/ sketch</td><td>- Implemented Dijkstra's Algorithm in Java (LandslideEvacuation.java) using adjacency list + min-priority queue with lazy deletion.
- Developed 3 test scenarios: (A) base 9-edge network, (B) additional road closure (edge 1→3 removed), (C) village isolated (no path).
- Verified all scenario outputs against manual step-by-step trace from Section 3.3 of the portfolio.
- Confirmed Scenario A optimal path: 0→1→3→5→7 = 55 minutes.
- Confirmed Scenario B auto-reroutes to 0→1→2→4→6→7 = 62 minutes.
- Confirmed Scenario C correctly reports no path and recommends helicopter.</td></tr>
<tr><td>Role</td><td><table>
<tr><td>Member 1</td><td>Member 2</td><td>Member 3</td><td>Member 4</td></tr>
<tr><td>DANIAL</td><td>KHAIROL</td><td>SHAH</td><td>IQBAL</td></tr>
<tr><td>Java implementation (core algorithm)</td><td>Pseudocode writeup</td><td>Test scenario design & edge cases</td><td>Manual execution trace verification</td></tr>
</table></td></tr>
</table>

<table>
<tr><td>Milestone 4</td><td>Correctness and time complexity analysis</td></tr>
<tr><td>Date (week)</td><td>WEEK 13 26 June 2026</td></tr>
<tr><td>Description/ sketch</td><td>- Completed Loop Invariant correctness proof (Initialisation → Maintenance → Termination → Path Reconstruction).
- Wrote full time complexity analysis for all components: O(E) + O(V) + O((V+E)logV) + O(V) = O((V+E)logV).
- Derived best/average/worst case analysis — all Θ((V+E)logV) since no early exit exists.
- Documented space complexity O(V+E).
- Completed all 15 portfolio markdown files and uploaded to GitHub.
- Completed presentation slide deck (10 slides) covering all required components.</td></tr>
<tr><td>Role</td><td><table>
<tr><td>Member 1</td><td>Member 2</td><td>Member 3</td><td>Member 4</td></tr>
<tr><td>DANIAL</td><td>KHAIROL</td><td>SHAH</td><td>IQBAL</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>Correctness proof (loop invariant)</td><td>Time complexity analysis (all cases)</td><td>Space complexity & algorithm comparison</td><td>Portfolio markdown completion & GitHub upload</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td>Milestone 5</td><td>Scenario Development</td></tr>
<tr><td>Date (week)</td><td>WEEK 14 (2 July 2026)</td></tr>
<tr><td>Description/ sketch</td><td>- Rehearsed and delivered 15-minute presentation to class.
- Each member presented their assigned sections clearly.
- Submitted: GitHub portfolio link, zipped Java source code, and completed project log.</td></tr>
<tr><td>Role</td><td>
<table>
<tr><td>Member 1</td><td>Member 2</td><td>Member 3</td><td>Member 4</td></tr>
<tr><td>DANIAL</td><td>KHAIROL</td><td>SHAH</td><td>IQBAL</td></tr>
<tr><td>Presentation slides design and Presentation delivery for Section 4 (Correctness & Complexity)</td><td>Presentation delivery for Sections 1 & 2 (Scenario, Algorithm Review)</td><td>Presentation delivery for Section 3 (Algorithm Design & Implementation)</td><td>GitHub portfolio upload and Final zip & Putrablast submission</td></tr>
</table>
</td></tr>
</table>